FEAT-ZD-48600 · DE-05 Data Model Changes · Data Model change for the feature

Data Engineering agent · item feat.data_model_change · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/feat.data_model_change.md · approved by Data engineer sim.data_engineer

feat.data_model_change: FEAT-ZD-48600

Data Model change for the Data Engineering lane of FEAT-ZD-48600 ("Show which overdue invoices belong to vendors that usually pay late"). This is DE-05, Data Model Changes, attempt 1. The workbook action is "Update the data model based on the source systems and ingested data". There is no gate. Automated checks, the independent judge and sampled human evaluation review this output (record:brief).

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-05 Data Model Changes, attempt 1	record:brief
Inputs	task.components (e526c4e, from DE-04), org.data_model (3b03f15)	record:brief; mcp:hobasa-context (context_list)
Background read	journey:feat.design (DES-02, revision 722a98f: DC-1, DC-2, B1.2 to B1.4, DD-6, DD-7, NF-2, NF-4)	journey:feat.design; mcp:hobasa-context (context_get)
Code read	ap_pipeline source and tests, from the step's tool view	tool:repo
Source system	Vendor invoice feed (CSV with a header row). It is v2 today; v3 adds the optional `paid_date` column	task.components (IN-1, section 2); org.data_model
Result	**The registered Invoice entity already holds `paid_date` (date or None), so no Invoice field is added, removed or retyped.** This step checks each field against the feed and the code. It records the two things the registered model does not yet describe: the reject reason value `paid_date` on the Rejected row, and the derived Delay history structure that `delays_by_vendor` produces. No storage, database or schema migration is involved	org.data_model; task.components (IN-3, IN-7, IN-8, section 7); journey:feat.design (A1, B1.3)

1. Baseline: the registered model compared with the code today

Element	Registered model (org.data_model)	Code today (`transform.to_model`)	Gap	Source
Invoice `id`	str	`r["invoice_id"].strip()`	None	org.data_model; tool:repo (`transform.py`)
Invoice `vendor`	str	`r["vendor"].strip()`	None	org.data_model; tool:repo (`transform.py`)
Invoice `due_date`	date	`date.fromisoformat(r["due_date"])`	None	org.data_model; tool:repo (`transform.py`)
Invoice `amount`	Decimal, 2 dp	`Decimal(r["amount"])`	None	org.data_model; tool:repo (`transform.py`); record:memory (D-2026-044)

Element	Registered model (org.data_model)	Code today (`transform.to_model`)	Gap	Source
Invoice `paid`	bool	`r.get("paid", "false").strip().lower() == "true"`	None	org.data_model; tool:repo (`transform.py`)
Invoice `paid_date`	date or None, "added in feed v3"	**Not produced.** The module docstring lists only `id, vendor, due_date, amount, paid`	Registered but not implemented. IN-3 (TSK-01) adds it in the Data Engineering coding step	org.data_model; tool:repo (`transform.py`); task.components (IN-3)
Rejected row	Not described	`{"row": r, "reason": type(e).__name__}`	`reason` gains the value `paid_date` (section 2.3)	tool:repo (`transform.py`); task.components (IN-3)
Delay history	Not described	Does not exist	New derived structure (section 2.4)	task.components (IN-7); journey:feat.design (DC-2, B1.3)

Package version today: `0.4.0` (tool:repo, `__init__.py`; task.components EX-7).

2. Data model changes

2.1 Source contract: vendor invoice feed row (v2 to v3)

Column	v2	v3	Type in the feed	Rule	Source
`invoice_id`, `vendor`, `due_date`, `amount`, `paid`	Present	Present, unchanged	str (CSV text)	Unchanged	task.components (section 2)
`paid_date`	Absent	**Added, optional**	ISO date string, or empty when unpaid	The key is missing in a v2 row and is `None` in a short v3 row. Ingestion needs no code change because `parse_feed` keys rows by header text	task.components (section 2, IN-2); journey:feat.design (B1.3 "Feed row")

2.2 Entity Invoice (DC-1): no field beyond the registered model

Field	Type	Nullable	Derived from the feed row	Change	Source
`id`	str	No	`invoice_id`, stripped	None	org.data_model; tool:repo
`vendor`	str	No	`vendor`, stripped	None	org.data_model; tool:repo
`due_date`	date	No	ISO date; no `strip()` is added	None	org.data_model; task.components (rules carried for coding)
`amount`	Decimal, 2 dp	No	`Decimal(...)`, never float	None	org.data_model; record:memory (D-2026-044, L-011)
`paid`	bool	No	`paid` text equals `true` (any case); missing means false	None	org.data_model; tool:repo

Field	Type	Nullable	Derived from the feed row	Change	Source
`paid_date`	date	**Yes (None)**	`None` when the value is missing, `None`, blank or whitespace only. Otherwise `date.fromisoformat(value.strip())`. A non-blank invalid value rejects the row	**Implemented with IN-3** (already registered)	org.data_model; task.components (IN-3); journey:feat.design (B1.2 `_parse_paid_date`, B1.4)

Rules for the record:

- The record stays a plain `dict`, not a class. Every existing key keeps its name and type. `paid_date` is present on **every** accepted record, and is `None` for feed v2 rows (journey:feat.design DC-1, B1.5 `test_feed_v2_without_paid_date_column`; task.components rules carried for coding).
- `paid_date` is validated only after the existing field checks pass, so an existing reject reason wins over `paid_date` (journey:feat.design DD-6, pending TQ-10; task.components IN-3, R-I2).
- An unpaid record may carry a valid `paid_date`, and the date is kept. This is an interim rule pending OQ-B6 (journey:feat.design DD-7; task.components CQ-I3).
- A paid record may have `paid_date = None`. It is kept, not rejected, and left out of the Delay history (journey:feat.design B1.4).
- No rule requires `paid_date` to be on or after `due_date`. An earlier date gives a negative delay, which is kept (journey:feat.design DC-2; task.components IN-7).

2.3 Rejected row: new reason value

Field	Type	Change	Source
`row`	dict (the feed row)	None	tool:repo (`transform.py`); journey:feat.design (B1.3)
`reason`	str	**New value** `paid_date` (constant `PAID_DATE_REASON`), alongside today's exception type names (`KeyError`, `ValueError`, `InvalidOperation`, `AttributeError`)	task.components (IN-3); tool:repo (`transform.py`)

The data-quality report needs no change. `reject_reasons` is the sorted set of every rejected row's `reason`, so `paid_date` appears in it automatically (task.components IN-4; tool:repo `dq.py`).

2.4 New derived structure: Delay history (DC-2)

Attribute	Definition	Source
Producer	`delays.delays_by_vendor(records)`, in a new pure module with no I/O	task.components (IN-7); journey:feat.design (B1.2)
Shape	`Dict[str, List[int]]`. The key is the Invoice `vendor`; the value is that vendor's list of `delay_days`	task.components (IN-7); journey:feat.design (DC-2, B1.3)
`delay_days`	`(paid_date - due_date).days`, in calendar days, as an int that may be negative	task.components (IN-7); journey:feat.design (NF-2, DC-2); record:memory (D-2026-031)
Membership	Only records where `paid` is truthy **and** `paid_date` is not `None`. A vendor with no such record has no key	journey:feat.design (B1.2, B1.5)

Attribute	Definition	Source
Order	Values in feed order; vendors in first-seen order	task.components (IN-7); journey:feat.design (B1.2)
Input not modified	The record list is not changed	task.components (IN-7); journey:feat.design (B1.2)
Persistence	None: in memory only, with no storage, database or schema migration	task.components (IN-8); journey:feat.design (A1)
Consumer	ap_forecast: one vendor's list is the `history` argument of `vendor_risk`	journey:feat.design (DC-2)

2.5 Knowledge-graph relations added for this feature (ap_pipeline side)

From	Relation	To	Via	Source
Feed row (v3)	maps to	Invoice (with `paid_date`)	`transform.to_model`	task.components (IN-3)
Feed row (v3)	rejected as	Rejected row (`reason = paid_date`)	`transform.to_model`	task.components (IN-3)
Rejected row	summarised in	Data-quality report `reject_reasons`	`dq.quality_report` (unchanged)	task.components (IN-4)
Invoice	aggregated by `vendor` into	Delay history	`delays.delays_by_vendor`	task.components (IN-7); journey:feat.design (DC-2)
Delay history	consumed by	ap_forecast `vendor_risk`	DC-2	journey:feat.design (A2, DC-2)
Invoice	consumed by	ap_aging `overdue_with_risk`, which reads `id`, `vendor`, `due_date`, `amount` and `paid`, and ignores `paid_date`	DC-1	journey:feat.design (DC-1, B3.3)

3. Compatibility

Check	Result	Source
Feed v2 (no `paid_date` column) still loads	Yes. Every record gets `paid_date = None`	task.components (section 2, R-I1); journey:feat.design (NF-4)
Existing keys, types and return shapes	Unchanged. The `to_model` signature and the `(records, rejected)` shape are kept	task.components (IN-3, rules carried for coding); journey:feat.design (NF-4)
Downstream consumer of Invoice (ap_aging)	Unaffected; it ignores the extra `paid_date` key	journey:feat.design (B3.3)
Monetary type	`amount` stays `Decimal`	record:memory (D-2026-044, L-011)
Schedule and pipeline orchestration	Unchanged: `pipeline.run` and `schedule.json`	task.components (IN-5, IN-6)

4. Schema validation rules (for DE-09 Unit Test Cases)

Rule	Expected	Source
The keys of every accepted record are exactly `id`, `vendor`, `due_date`, `amount`, `paid`, `paid_date`	Pass	org.data_model; journey:feat.design (DC-1)
`paid_date` is a `date` or `None`	Pass	org.data_model
Blank or missing `paid_date`	`None`; the row is accepted	journey:feat.design (B1.4)
Non-blank invalid `paid_date` (`"2026-13-01"` or `"not-a-date"`) on an otherwise valid row	Rejected with reason `paid_date`, which `reject_reasons` lists	task.components (rules carried for coding); journey:feat.design (B1.4, B1.5)

Rule	Expected	Source
Bad `due_date` and bad `paid_date` together	Reason `ValueError`	journey:feat.design (B1.4, B1.5, DD-6)
Unpaid row with a valid `paid_date`	Accepted; the date is kept	journey:feat.design (DD-7, B1.5)
Delay history	Values are `int`. Keys are only vendors with at least one paid record that has a `paid_date`	journey:feat.design (DC-2, B1.5)
`amount`	`Decimal`, never `float`	record:memory (D-2026-044)

5. Code in this step

This step changes the data model only and changes no code. The tool view shows `transform.py` without `paid_date`, and there is no `delays.py` module yet (tool:repo). IN-3 and IN-7 implement the model change in DE-08 Coding, within the file budget in task.components section 4. The CI command `/usr/bin/python3 ci.py` was run on the unchanged `feature/story` checkout in this step and passed, and `git status` showed a clean working tree (record:brief for the command; result from this step's CI run). Agent judgement: the `transform.py` module docstring lists the model fields, so it should gain `paid_date` with IN-3 (tool:repo `transform.py`).

6. Open questions (non-blocking)

ID	Question	Interim answer	Owner (proposed)
DMQ-1	Should the registered org.data_model be extended with the Rejected row `reason` value `paid_date` and the Delay history structure (sections 2.3 and 2.4)? Today it describes only the Invoice entity	Treat this document as the feature-level model change. Data modelers decide whether to register these items org-wide	Data engineering lead; data modelers
DMQ-2	Carried CQ-I1: is the feed v3 column spelled exactly `paid_date`?	Assume `paid_date`, as in org.data_model	Data engineering lead / feed owner
DMQ-3	Carried CQ-I3 / OQ-B6 (DD-7): should an unpaid record keep a valid `paid_date`?	Yes; the Delay history ignores it	Product / Architect
DMQ-4	Carried DQ-4: should `delays_by_vendor` be re-exported from `ap_pipeline/__init__.py`?	No. It is imported as `ap_pipeline.delays.delays_by_vendor`	Architect
DMQ-5	Carried TQ-10 (DD-6): does an existing reject reason win over `paid_date`?	Yes	Architect

7. Validation check

Check	Result	Source
Data model updated from the source system and the ingested data	Pass: feed v3 contract (2.1), Invoice (2.2), Rejected row (2.3), Delay history (2.4) and relations (2.5)	task.components; org.data_model
Conflict with the registered model or an approved output	None. `paid_date` matches org.data_model and DC-1, and the additions are new structures, not changes	org.data_model; journey:feat.design; task.components (section 9)
Production or source-system data read	None. Only the inputs, the tool view and synthetic test rules were used	record:brief
Blocking questions	None	n/a